1 3D Geometry Processing Overview

In the context of *mesh and geometry processing*, the polygonal mesh represents one of the most widespread data structures for surface discretization. Indeed, it allows for the representation of continuous surfaces through a partition of the continuous space into polygonal cells of arbitrary size [1]. Formally, a 3D mesh is defined as a pair M=(V,F), where V and F are sets defined as follows:

- $\mathcal{V} = \{\mathbf{v} \in \mathbb{R}^3\}$ is the set of vertices in three-dimensional Euclidean space;
- $\mathcal{F} \subseteq \{\{i, j, k\} \subset \mathbb{N} \mid 1 \leq i, j, k \leq |\mathcal{V}|\}$ is the set of faces, described by triplets of indices referring to the vertices.

This representation is extremely useful in *geometry processing* and computer graphics, as it explicitly preserves the topological information of the geometric structure and represents the natural output format for most three-dimensional acquisition procedures (e.g., laser scanning, photogrammetry).

To characterize the geometric and topological structure of a mesh $\mathcal{M}$ and evaluate its suitability for the application of standard algorithms without encountering singularities or numerical errors, the following fundamental properties are defined:

- **Manifoldness:** A mesh $\mathcal{M}$ is defined as *2-manifold* if every point on its surface has a neighborhood homeomorphic to an open disk in $\mathbb{R}^2$ (or a half-disk for boundary points). Combinatorially, for a polygonal mesh, this requires that:
  1. Every edge is shared by at most two faces;
  2. The set of faces incident to any vertex constitutes a single connected fan (or semi-fan for boundary vertices), excluding degenerate configurations such as hourglass vertices.
- **Orientability:** A manifold mesh is *orientable* if it is possible to define a vertex traversal order for each face such that the resulting normals are consistent across the entire surface. Formally, for every pair of adjacent faces f_i, f_j sharing an edge e, the orientation induced on e by f_i must be opposite to that induced by f_j. If such consistency cannot be guaranteed globally (e.g., Möbius strip), the mesh is non-orientable.
- **Boundary Characterization (Closed vs. Open):** This property classifies the mesh based on the presence of the boundary $\partial\mathcal{M}$.
  - A mesh is *closed* (or *watertight*) if $\partial\mathcal{M} = \emptyset$, meaning every edge is shared by exactly two faces. A closed and orientable mesh partitions the $\mathbb{R}^3$ space into two distinct regions (interior and exterior).
  - A mesh is *open* (or *with boundary*) if there exists a non-empty set of edges incident to only one face.

1.1 Simplification via Quadratic Error Metrics

In the field of *mesh processing*, simplification (or decimation) represents a critical operation for managing geometric complexity. Given a discrete model $\mathcal{M} = (\mathcal{V}, \mathcal{F})$, this process aims to generate an approximation $\mathcal{M}' = (\mathcal{V}', \mathcal{F}')$ such that $|\mathcal{V}'| < |\mathcal{V}|$, optimizing the *trade-off* between complexity and geometric fidelity. Formally, the problem can be modeled according to two dual approaches:

[1] For simplification, we assume polygons with triangular faces, although quad-meshes and polygonal-meshes exist.

- **Error minimization (*Fixed budget*):** determine $\mathcal{M}'$ such that the geometric deviation $\|\mathcal{M} - \mathcal{M}'\|$ is minimal, subject to the constraint $|\mathcal{V}'| = k$ (where k is the target number of vertices);
- **Complexity minimization (*Error bounded*):** determine $\mathcal{M}'$ with minimum cardinality $|\mathcal{V}'|$, ensuring that the approximation error is limited by a threshold, $\|\mathcal{M} - \mathcal{M}'\| < \varepsilon$.

In the formulation presented above, the notation $\|\mathcal{M} - \mathcal{M}'\|$ denotes a measure of dissimilarity between the surfaces. In the literature, the standard metric to rigorously quantify such geometric deviation is the Hausdorff Distance [ASCE02], defined as the maximum of the minimum distances between the points of the two sets.

Although the literature proposes various simplification strategies, the prevailing approach in the state of the art is based on *iterative edge contraction*. In-depth comparative studies [CMS98] have highlighted the superiority of this class of algorithms, which operate through an iterative decimation process guided by a local error metric. In this procedure, the system evaluates the cost associated with the collapse of each edge, selecting at each step the operation that induces the minimum geometric distortion. Within this paradigm, the algorithm based on *Quadratic Error Metrics* (QEM), introduced by Garland and Heckbert [GH97], has established itself as the reference standard.

1.2 Quadric Error Metrics Algorithm

The proposed simplification algorithm is based on the iterative contraction of *vertex pairs*, guided by a quadratic error metric that allows for quantifying and minimizing the geometric distortion introduced by each decimation operation.

1.2.1 Error Approximation

To evaluate the cost of a contraction, a 4×4 symmetric matrix, denoted by $\mathbf{Q}$, is associated with each vertex $\mathbf{v} = [v_x, v_y, v_z, 1]^T$. The local error at vertex $\mathbf{v}$ is defined as the quadratic form:

$$\Delta(\mathbf{v}) = \mathbf{v}^T \mathbf{Q} \mathbf{v}$$

When a pair of vertices $(\mathbf{v}_1, \mathbf{v}_2)$ is contracted into a new vertex $\bar{\mathbf{v}}$, the error matrix associated with $\bar{\mathbf{v}}$ is calculated according to an additive rule:

$$\bar{\mathbf{Q}} = \mathbf{Q}_1 + \mathbf{Q}_2$$

This rule ensures that the error accumulated in the new vertex approximates the sum of the original errors.

To determine the optimal position of $\bar{\mathbf{v}}$ that minimizes the local error, the point where the gradient of the error function is zero ($\nabla\Delta(\bar{\mathbf{v}}) = 0$) is sought. Since Δ is quadratic, the problem reduces to solving the following linear system:

$$\begin{bmatrix} q_{11} & q_{12} & q_{13} & q_{14} \\ q_{12} & q_{22} & q_{23} & q_{24} \\ q_{13} & q_{23} & q_{33} & q_{34} \\ 0 & 0 & 0 & 1 \end{bmatrix} \bar{\mathbf{v}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

If the coefficient matrix is invertible, the system provides the position that minimizes the error; in case of singularity, the algorithm evaluates alternative positions along the segment $\mathbf{v}_1\mathbf{v}_2$ (the endpoints or the midpoint).

1.2.2 Geometric Derivation of Quadrics

The construction of the initial $\mathbf{Q}$ matrices is based on the observation that, in the original model, each vertex lies on the intersection of the planes defined by the incident triangles. The error is therefore formulated as the sum of the squares of the distances of the vertex from that set of planes:

$$\Delta(\mathbf{v}) = \sum_{\mathbf{p}\in\text{planes}(\mathbf{v})} (\mathbf{p}^T\mathbf{v})^2$$

Where $\mathbf{p} = [a, b, c, d]^T$ represents the coefficients of the plane equation $ax + by + cz + d = 0$ (with $a^2 + b^2 + c^2 = 1$). This expression can be rewritten in matrix form:

$$\Delta(\mathbf{v}) = \sum_{\mathbf{p}\in\text{planes}(\mathbf{v})} \mathbf{v}^T(\mathbf{p}\mathbf{p}^T)\mathbf{v} = \mathbf{v}^T \left(\sum_{\mathbf{p}\in\text{planes}(\mathbf{v})} \mathbf{K}_p \right) \mathbf{v}$$

Where $\mathbf{K}_p = \mathbf{p}\mathbf{p}^T$ is the *fundamental quadric* associated with a single plane. Consequently, the initial $\mathbf{Q}$ matrix for each vertex is simply the sum of the fundamental matrices of the incident planes.

1.2.3 Algorithm Summary

In summary, the proposed simplification procedure is organized into an iterative process based on the contraction of *vertex pairs*, guided by the minimization of the local quadratic error. The operational sequence is described as follows:

1. **Quadric Initialization:** For each vertex of the initial mesh, the matrix Q is calculated by summing the fundamental quadrics of the incident planes.

2. **Pair Selection:** All valid vertex pairs (v_1, v_2) susceptible to contraction (typically coinciding with the mesh edges) are identified.

3. **Target and Cost Calculation:** For each valid pair (v_1, v_2), the aggregated matrix $\bar{\mathbf{Q}} = \mathbf{Q}_1 + \mathbf{Q}_2$ is calculated and the optimal position of the contracted vertex $\bar{\mathbf{v}}$ is determined by solving the linear system $\nabla\Delta(\bar{\mathbf{v}}) = 0$. The associated contraction cost is defined as the error at that point:

   $$\text{Cost}(v_1, v_2) = \bar{\mathbf{v}}^T\bar{\mathbf{Q}}\bar{\mathbf{v}}$$

   Each edge is inserted into a priority queue sorted by minimum cost.

4. **Iterative Contraction:** The pair (v_1, v_2) with the minimum cost is extracted from the queue and the contraction $(v_1, v_2) \to \bar{\mathbf{v}}$ is performed, removing v_1 and v_2 and updating the local topology 1.

5. **Update:** Costs are recalculated for all valid pairs incident to the new vertex $\bar{\mathbf{v}}$, and their position in the priority queue is updated.

6. **Termination:** The procedure is repeated (steps 5-6) until the preset stopping criterion is met (e.g., target number of faces or maximum error threshold).

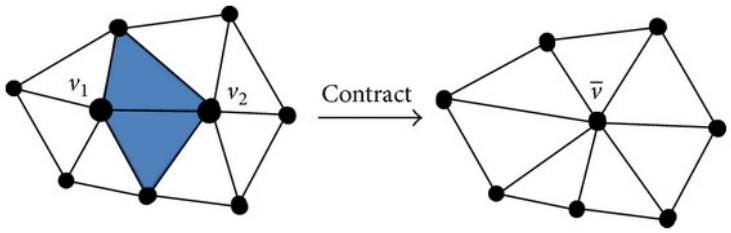


Figura 1: Schematic representation of the *edge contraction* operation. The highlighted edge (v_1, v_2) is collapsed into a single optimal vertex $\bar{\mathbf{v}}$. The faces incident to the edge (shaded) become degenerate and are removed from the topology, reducing the local complexity of the mesh [GH97].

In the proposed solution, a **Manifold** mesh is assumed, and the presence of holes is allowed.

2 Shared Memory Parallel Implementation

As discussed in Section [1.2], the standard QEM algorithm is inherently sequential in nature. The decimation process, based on a global priority queue, implies that the state of the mesh at iteration n strictly depends on the topological changes made at iteration $n-1$. Consequently, a direct parallelization of the main loop is complex. Although the initialization phase (calculation of initial quadrics and collapse costs) is trivially parallelizable by dividing vertices and edges into independent *chunks*, performance analysis highlights that this phase has a limited impact on the total execution time, thus restricting the possible overall *speedup* (Amdahl's Law).

To overcome this limitation, a **data-parallel** approach based on spatial partitioning of the domain has been adopted. The mesh $\mathcal{M}$ is subdivided into disjoint sub-meshes, assigned to distinct threads that perform the local simplification operation concurrently. At the end of the parallel phase, the partitions are rejoined in a *merge* phase, followed optionally by a sequential *refinement* step to satisfy the global stopping criteria.

2.1 Boundary Management

The main challenge of this architecture lies in managing data consistency along the partition boundaries (*partition boundaries*). Since boundary elements are shared between adjacent threads, the concurrent extraction and modification of a boundary *edge* from the local priority queue would introduce *race conditions* and topological inconsistency.

While it is theoretically possible to implement synchronization and inter-thread communication mechanisms to handle shared updates, such an approach would introduce considerable synchronization *overhead* and high logical complexity. Therefore, the adopted implementation strategy involves **boundary locking** [2]: vertices and edges located on the cut lines are marked as immutable during the parallel phase. These elements are processed exclusively in a second post-merge refinement phase, thereby ensuring the structural integrity of the mesh without penalizing parallel computing efficiency.

In detail, the implemented procedure is based on a preliminary classification of geometric primitives (vertices, edges) through a binary status attribute, defined as *collapsable* or *non-collapsable*. This attribute acts as a discriminant to determine the eligibility of the element for the parallel decimation process: only primitives internal to a partition and not incident on the boundaries are processed, ensuring the absence of write conflicts.

- **Partition Indexing:** A unique identifier (*Partition ID*) is assigned to each cell or subdomain generated by the spatial partitioning strategy.
- **Vertex-Cell Association:** For each vertex $\mathbf{v} \in \mathcal{V}$, the identifier of its belonging spatial cell is calculated and stored. [2]
- **Boundary Detection:** The set of edges $\mathcal{E}$ is analyzed. For each edge $e = (v_i, v_j)$, the cell identifiers of the endpoint vertices are compared. If these identifiers differ, the edge intersects the boundary between two partitions; consequently, both vertices v_i and v_j are marked as *non-collapsable* (locked) to preserve the integrity of the interface between chunks.

[2] It is assumed that, by virtue of the adopted partitioning technique (e.g., uniform grid), this spatial mapping operation can be performed with constant time complexity $O(1)$.

- **Constraint Propagation:** Finally, a consistency pass is performed on the edges. Any primitive that is incident on (or contains) at least one vertex marked as *non-collapsable* inherits the immovable status, being excluded from the current parallel simplification phase.

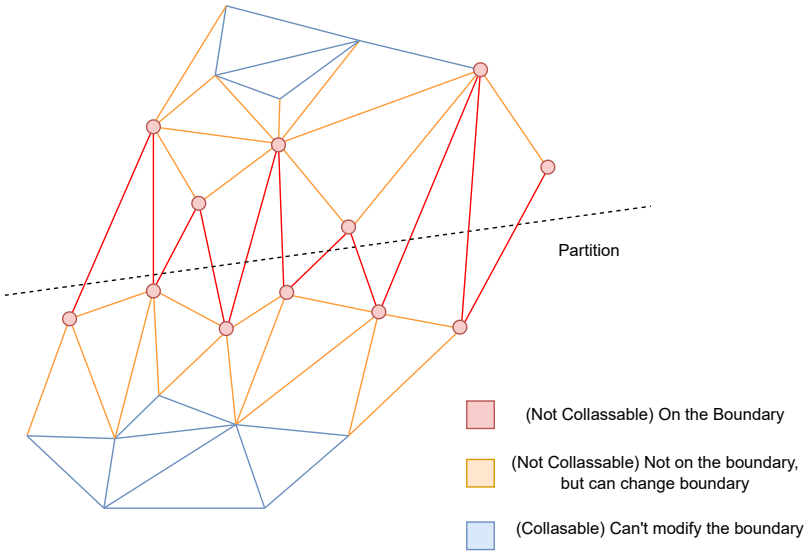


Figura 2: Schematic representation of the *boundary locking* mechanism. The black dashed line indicates the logical boundary between two partitions. Edges crossing this boundary make the incident vertices "locked" (represented in red).

2.2 Spatial Partitioning

Another critical aspect for the success of a parallel implementation lies in the spatial domain partitioning strategy. This mechanism must ensure optimal *load balancing*, distributing the computational load equitably among the available threads, while minimizing the *overhead* introduced both in the construction phase of the spatial data structure and in the subsequent reassembly (*merging*) phase of the partial results.

To evaluate the impact of spatial subdivision on the performance of the simplification algorithm, two distinct data structures were implemented and compared: one based on a **Uniform Grid** and one based on an **Octree**.

2.2.1 Uniform Grid

The *Uniform Grid* [3] represents the most direct approach to spatial subdivision. In this configuration, the global *Bounding Box* of the mesh is discretized into a regular grid of cubic or parallelepiped cells of fixed size.

Given a vertex $\mathbf{v} = (x, y, z)$ and defined the cell size $\mathbf{d} = (d_x, d_y, d_z)$, the belonging cell index (i, j, k) can be calculated with $O(1)$ complexity through a direct mapping:

$$i = \left\lfloor \frac{x - x_{min}}{d_x} \right\rfloor, \quad j = \left\lfloor \frac{y - y_{min}}{d_y} \right\rfloor, \quad k = \left\lfloor \frac{z - z_{min}}{d_z} \right\rfloor$$

The main advantages of this solution lie in the speed of construction and access, which scales linearly with the number of vertices ($O(N)$), and the simplicity of data locality management, which favors

cache usage. However, the Uniform Grid proves inefficient in the presence of meshes with non-uniform geometric distribution (e.g., CAD models with large empty areas and concentrated details), risking the generation of unbalanced partitions (some empty cells, others overloaded), thus compromising parallelism.

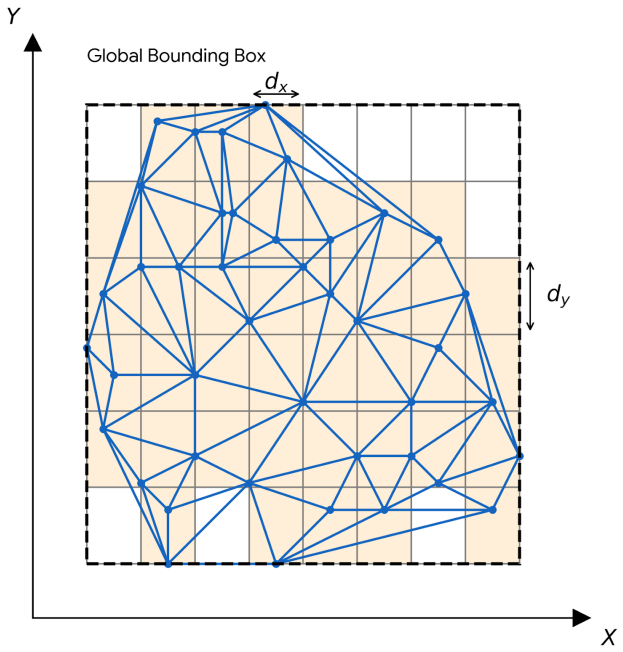


Figura 3: Visualization of the *Uniform Grid*. Each active cell (in orange) contains a subset of geometric primitives, allowing efficient local access during the parallel processing phase.

2.2.2 Octree

The *Octree* [4] offers a hierarchical and adaptive partitioning strategy. Unlike the uniform grid, the space is recursively subdivided into eight octants only where geometric density requires it. The construction process begins by considering the entire volume of the scene as the root node and proceeds by subdividing nodes that contain a number of primitives higher than a preset threshold or until a maximum depth is reached.

This structure ensures better load balancing even for highly irregular geometries, as the cells (the leaves of the tree) tend to contain a similar number of elements. Conversely, the use of the Octree introduces a greater computational *overhead*:

- The construction complexity is $O(N \log N)$;
- Traversing the structure to identify neighbors and manage boundaries (*boundary locking*) is more expensive than direct grid access;
- The tree structure involves less contiguous memory usage, potentially increasing *cache misses*. Although this issue can be resolved in the case of static octrees.

Despite the higher theoretical complexity, the Octree proves to be more efficient than a uniform grid for obtaining a more balanced mesh subdivision [CMRS03].

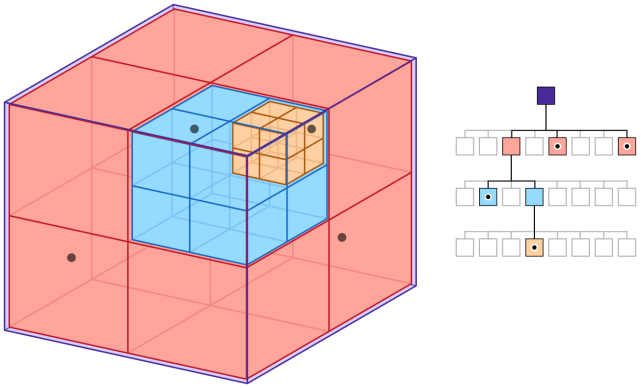


Figura 4: Conceptual representation of a hierarchical spatial partition via *Octree*. The root volume is recursively decomposed into eight octants (visualized with increasing depth levels: red, blue, orange) based on local data density.

2.3 Hierarchical Parallel Reduction

Currently, the parallel simplification pipeline consists of four distinct phases:

1. **Splitting:** subdivision of the geometric domain.
2. **Simplification:** assignment of cells to available threads and simplification.
3. **Merging:** aggregation of partial results and reconstruction of the topology.
4. **Sequential Refinement:** processing of previously locked boundary elements.

Although this architecture ensures a significant performance increase (with an observed *speedup* of about 6× compared to the sequential counterpart), it suffers from an intrinsic limitation related to scalability. As parallelism increases (i.e., the number of threads), it is necessary to increase the number of spatial partitions to maintain adequate load balancing. However, a larger number of partitions implies a growth in the density of *partition boundaries* and, consequently, in the number of geometric primitives "locked" to prevent *race conditions*. These primitives must be processed exclusively during the final sequential refinement phase, which becomes progressively more expensive, acting as a bottleneck according to Amdahl's law.

To mitigate this problem and improve scalability on *multi-core* architectures, a *Hierarchical Reduction* approach is proposed. The key idea consists of replacing the single parallel pass with an iterative sequence of k parallel passes, progressively reducing the granularity of the spatial partitioning.

However, a naive reduction strategy (e.g., a linear halving of cells at each step) would be ineffective, as it would tend to keep the spatial position of the main boundaries unchanged, leaving the same sets of vertices locked throughout the iterations. To maximize the unlocking of boundary vertices, it is necessary to introduce a shift (*shift*) of the cut lines between one step and the next. For this purpose, a reduction function has been formulated that alternates even and odd subdivisions:

$$n_{k+1} = f(n_k) = \begin{cases} 1 & \text{if } n_k \leq 4 \\ \frac{n_k}{2} + 1 & \text{if } n_k > 4 \wedge n_k \equiv 0 \pmod 2 \\ \lfloor \frac{n_k}{2} \rfloor & \text{if } n_k > 4 \wedge n_k \not\equiv 0 \pmod 2 \end{cases}$$

where n_k represents the number of subdivisions along an axis at iteration k. The $+1$ term in the even case introduces a deliberate asymmetry that misaligns the boundaries with respect to the previous

iteration, allowing threads to process previously locked elements. The function guarantees convergence to $n = 1$ (single global partition), ensuring that the algorithm terminates with a final phase in which the simplification target is definitely reached.

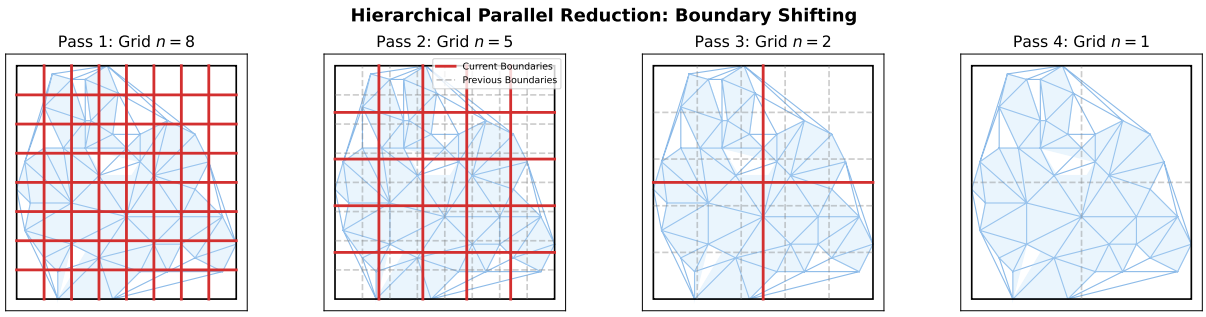


Figura 5: Evolution of the grid during *Hierarchical Parallel Reduction*. In each frame, the solid red lines indicate the active boundaries of the current partition (n_k), while the dashed gray lines represent the boundaries of the previous partition (n_{k-1}).

It is important to note that this dynamic reduction strategy is effectively applicable only in the context of the *Uniform Grid*. Using rigid hierarchical data structures like *Octrees* would make this method inapplicable, as their topology imposes fixed constraints on spatial subdivision (constant branching factor of 8), preventing the flexible *shifting* of boundaries necessary for the proposed optimization.

3 Distributed Memory Implementation

The shared memory parallel architecture, analyzed in Section 2, has proven capable of guaranteeing significant *speedup* increases compared to sequential counterparts, despite the computational overhead introduced by the partitioning, merging (*merge*) phases, and the management of boundary consistency to prevent *race conditions*. This efficiency stems from the ability to maintain the entire dataset in a single address space, allowing direct access to geometric primitives (vertices, faces, edges) via indexing and maximizing the effectiveness of the memory hierarchy by exploiting spatial and temporal locality principles (*cache locality*).

However, the dependence on centralized memory represents a structural constraint that limits system scalability in distributed *High Performance Computing* (HPC) contexts, where memory is physically fragmented across distinct compute nodes. To extend the algorithm's applicability to compute clusters, an architectural restructuring based on the hierarchical *Master-Worker* paradigm has been implemented. In this configuration, a designated node called the *Master* assumes the role of orchestrator and resource manager, while N *Worker* nodes execute the computational load in parallel on disjoint portions of the domain. The distributed execution flow is as follows:

1. **I/O and Pre-processing (Master):** The Master node loads the high-resolution mesh and initializes the global data structures.
2. **Coarse-Grained Partitioning (Master):** The geometric domain is decomposed into macro-partitions. Unlike the shared memory approach, a coarse granularity is preferred in this context to minimize the communication-to-computation ratio.
3. **Asynchronous Distribution (Master $\rightarrow$ Worker):** Partitions are serialized and sent to available Worker nodes via asynchronous (non-blocking) communication protocols, allowing communication phases to overlap with computation.
4. **Data Acquisition (Worker):** Each Worker receives its assigned sub-mesh and locally reconstructs the topology necessary for processing.
5. **Local Simplification (Worker):** The node executes the QEM simplification algorithm on its partition, adopting the local optimization logic and memory management already validated in the shared memory implementation.
6. **Result Transmission (Worker $\rightarrow$ Master):** Upon completion of the decimation, the simplified sub-mesh and its relative metadata are transmitted back to the Master node.
7. **Global Merge (Master):** The Master aggregates the partial results received from the various Workers, reconstructing the topological consistency of the global mesh.
8. **Final Refinement and Serialization (Master):** A final refinement phase is performed to resolve any discontinuities or artifacts along the cut boundaries of the macro-partitions, followed by the exportation of the simplified model to disk.

A crucial aspect of the distributed memory implementation lies in data partitioning management. Unlike the *shared memory* approach, where threads operate on a single virtual address space accessing primitives via indices, the distributed architecture prevents direct access to remote memory. Consequently, the distribution phase cannot be limited to a logical subdivision but requires a **physical copy**

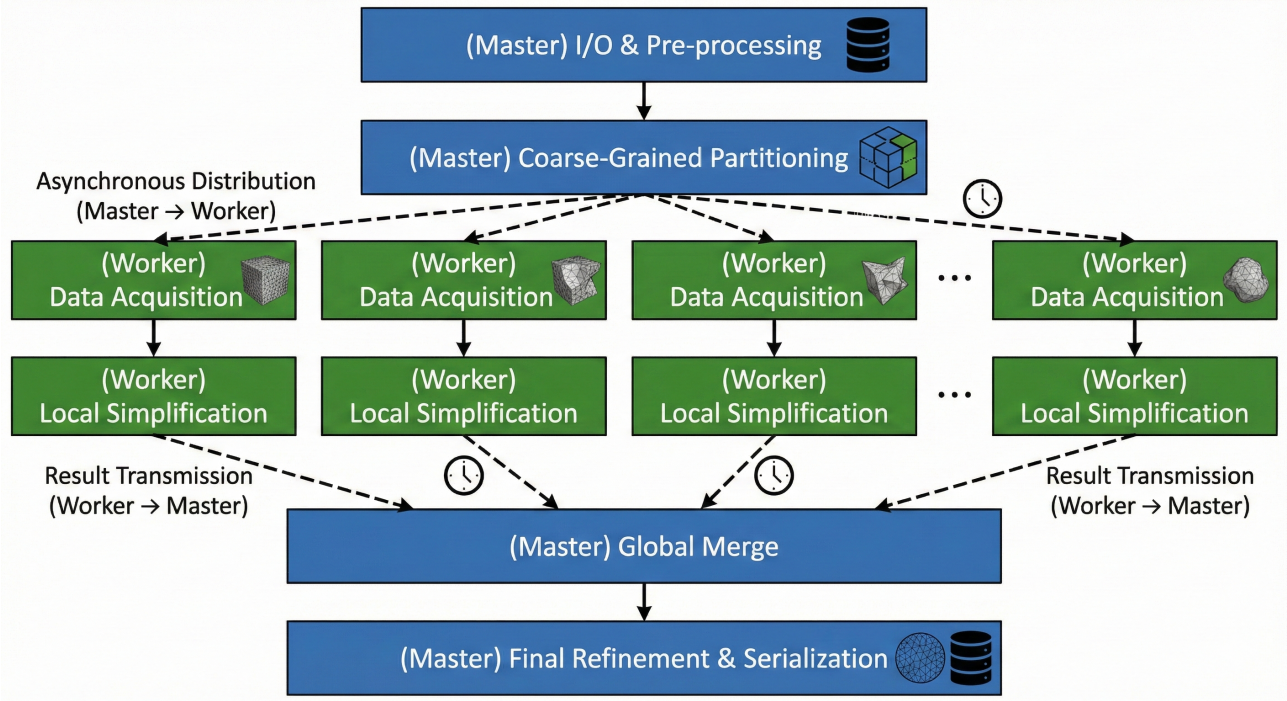


Figura 6: Flowchart of the distributed memory architecture based on the *Master-Worker* paradigm. The Master node orchestrates the workload by performing coarse-grained partitioning and asynchronous data distribution. Worker nodes process sub-meshes in parallel independently, returning simplified geometries that are subsequently aggregated (*Global Merge*) and refined centrally by the Master to ensure global topological consistency.

of the data: the Master must segregate the vertex and face arrays into N distinct subsets, serialize them, and transmit them to the respective Worker nodes.

In this context, the topology preservation strategy via *boundary locking* must be extended to manage a dual hierarchy of consistency:

- **Local Consistency (Intra-node):** Management of concurrency between threads operating within the same Worker node. The same *locking* logic seen in the shared memory implementation is applied.
- **Global Consistency (Inter-node):** Management of consistency along the cut interfaces separating sub-meshes assigned to different Workers. It is fundamental to prevent two distinct nodes from independently attempting to simplify a primitive located on a shared boundary, which would lead to topological divergences.

To implement this constraint, a **data replication** technique is adopted (often referred to as *Ghost Cells* or *Halo Regions*). During the partitioning phase, if a geometric primitive (vertex or face) lies on the intersection between two or more partitions, it is duplicated and sent to all involved Workers. During the local initialization phase, each Worker analyzes its sub-mesh: primitives belonging exclusively to the current node are classified as *global-collapsable* (then subject to local constraints), while replicated primitives (globally shared) are marked as *locked* a priori. This operation ensures that decimation operations occur in total isolation.

3.1 Pure Distributed Data-Parallel Approach

As an alternative to the hybrid model (MPI+OpenMP) previously discussed, this section proposes a *purely distributed* architecture oriented towards massive data-parallelism. In this configuration, every process within the cluster is treated as an independent single-threaded processing unit (*single-threaded worker*), eliminating the complexity of local shared-memory synchronization in favor of explicit inter-node communication management.

The substantial difference compared to the classic Master-Worker approach lies in the management of the merging and refinement phase. Instead of centralizing the *merging* on the Master node, a **Distributed Hierarchical Reduction** scheme is adopted.

The operational flow consists of the following phases:

1. **Fine-Grained Partitioning (Master):** The Master node decomposes the initial domain directly with the target granularity required to obtain an adequate simplification. Note that if N are the workers per partition, N can be greater than the number of workers.

2. **Scatter & Local Simplification:** Partitions are distributed to the workers. Each node W_i performs local simplification on partition P_i (or a set of partitions) independently, requiring no communication during this phase.

3. **Distributed Reduction Tree:** At the end of local simplification, workers do not return the result to the Master but start a staged reduction procedure. Assuming a binary reduction topology [3], at each stage k, workers are associated with specific cells of the higher-level grid:

   - Worker W_i (source) sends its simplified geometry to worker W_j (destination/aggregator), where W_j is responsible for the parent cell at level $k+1$.
   - The aggregator worker W_j, once it has received fragments from the child nodes (e.g., its own previous fragment and that of the neighbor), performs a local *Merge* operation.
   - Subsequently, W_j executes a new simplification pass on the unified geometry to respect the polygonal budget constraints of the current level.

4. **Convergence:** The process repeats recursively climbing the reduction tree. Nodes that have completed their transmission become inactive, while aggregator nodes continue processing. The algorithm terminates when the last root node (which may coincide with the Master or an elected worker) possesses the entire simplified mesh (single partition).

The exact working diagram of a worker can be seen in figure 7, which precisely describes all the stages a worker can be in.

This approach offers the theoretical advantage of distributing the computational cost of *merging* and boundary *refinement* across the entire cluster, theoretically reducing the I/O and memory bottleneck on the Master node. However, it introduces significant network latency due to the iterative transfer of partial geometries between workers.

3.1.1 Load Balancing and Spatial Mapping

A critical aspect in distributed architecture is the load balancing strategy; indeed, the mapping method between spatial grid cells and worker nodes drastically influences global performance by directly acting

[3] In the final implementation, as previously seen, a reduction that alternates even and odd phases is used to accentuate simplification on the boundaries.

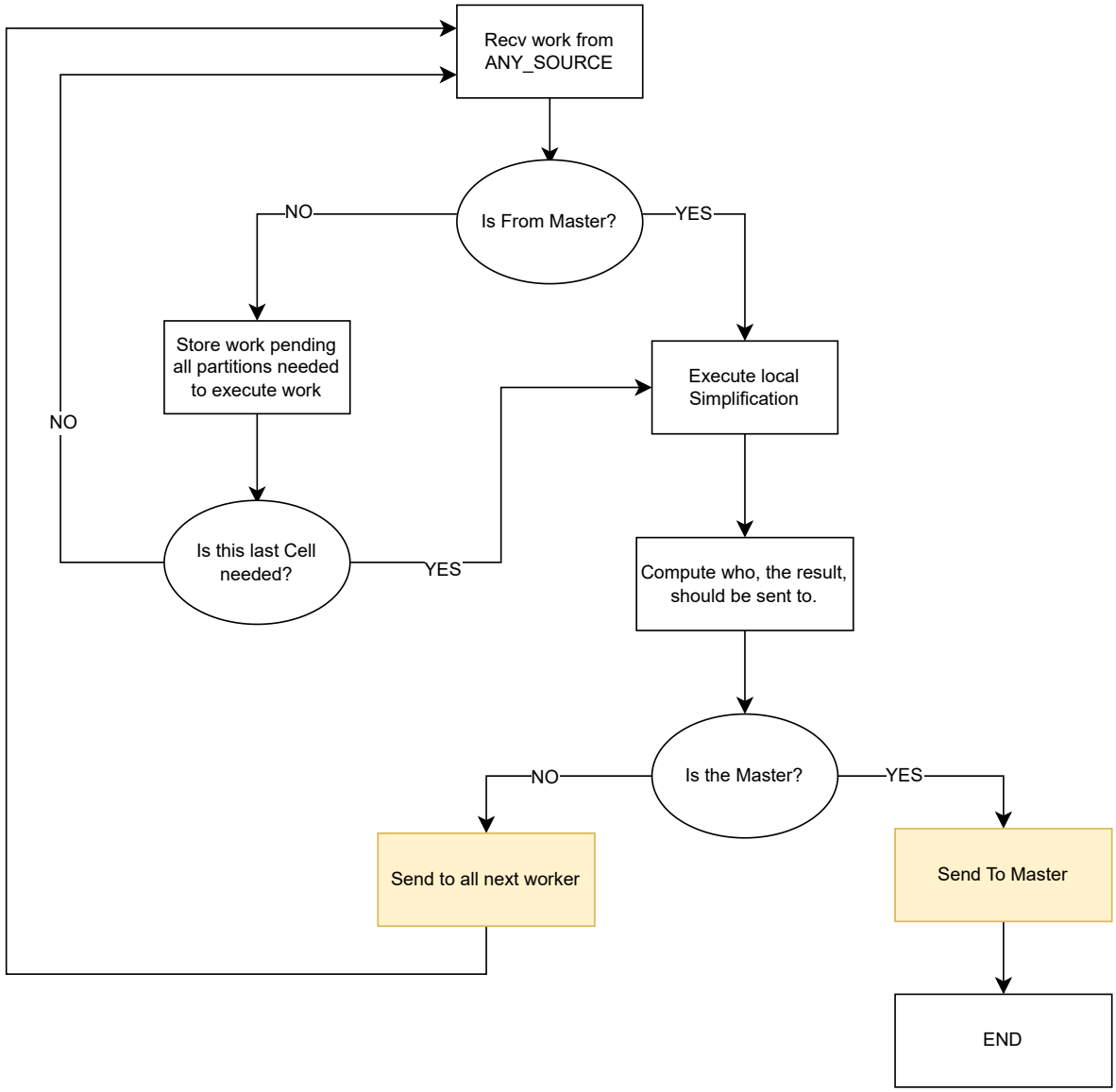


Figura 7: Flowchart of the operational logic of the Worker node in the *Pure Distributed Data-Parallel* architecture. Non-blocking stages, which use an asynchronous communication mechanism, are described in yellow.

on the volume of network traffic generated. The primary objective of the assignment algorithm is to maximize **spatial locality**: geometrically adjacent cells should ideally be assigned to the same worker or to contiguous workers. This approach reduces communication costs during the hierarchical reduction phase, as the *merging* of adjacent fragments residing on the same node resolves into local memory (RAM) copy operations.

The implemented mapping function adopts a cubic block decomposition strategy. Given a total number of workers W and a resolution grid S (split), the mapping of a generic cell with linear index i to the destination worker w_{dest} occurs through the following steps:

1. **Worker Topology Definition:** The arrangement of workers is approximated in a logical three-dimensional grid of size $P \times P \times P$, where P represents the number of partitions along an

axis:

$$P = \left\lfloor \sqrt[3]{W} \right\rfloor$$

2. **Block Sizing:** The geometric space is subdivided into macro-blocks of size B, determined by the grid resolution and worker partitioning:

$$B = \left\lceil \frac{S}{P} \right\rceil$$

3. **Spatial Mapping:** Given the three-dimensional coordinates of the cell $\mathbf{c} = (x, y, z)$ derived from the index i, the coordinates of the belonging macro-block $\mathbf{b} = (b_x, b_y, b_z)$ are identified:

$$b_k = \min\left(\left\lfloor \frac{c_k}{B} \right\rfloor, P - 1\right) \quad \text{for } k \in \{x, y, z\}$$

4. **Linearization and Assignment:** The base partition identifier ID_{part} is calculated by linearizing the block coordinates (row-major order):

$$ID_{part} = b_z P^2 + b_y P + b_x$$

In the standard case where $W \leq P^3$, the assignment is direct:

$$w_{dest} = ID_{part} \pmod{W}$$

However, should the number of available workers exceed the capacity of the cubic topology ($W > P^3$), the algorithm employs a refinement (sub-partitioning) strategy. The macro-block is further subdivided among K workers assigned to that specific spatial partition, calculating a local rank r based on the cell's position within the block itself. The general assignment formula thus becomes:

$$w_{dest} = \left(ID_{part} + r \cdot P^3\right) \pmod{W}$$

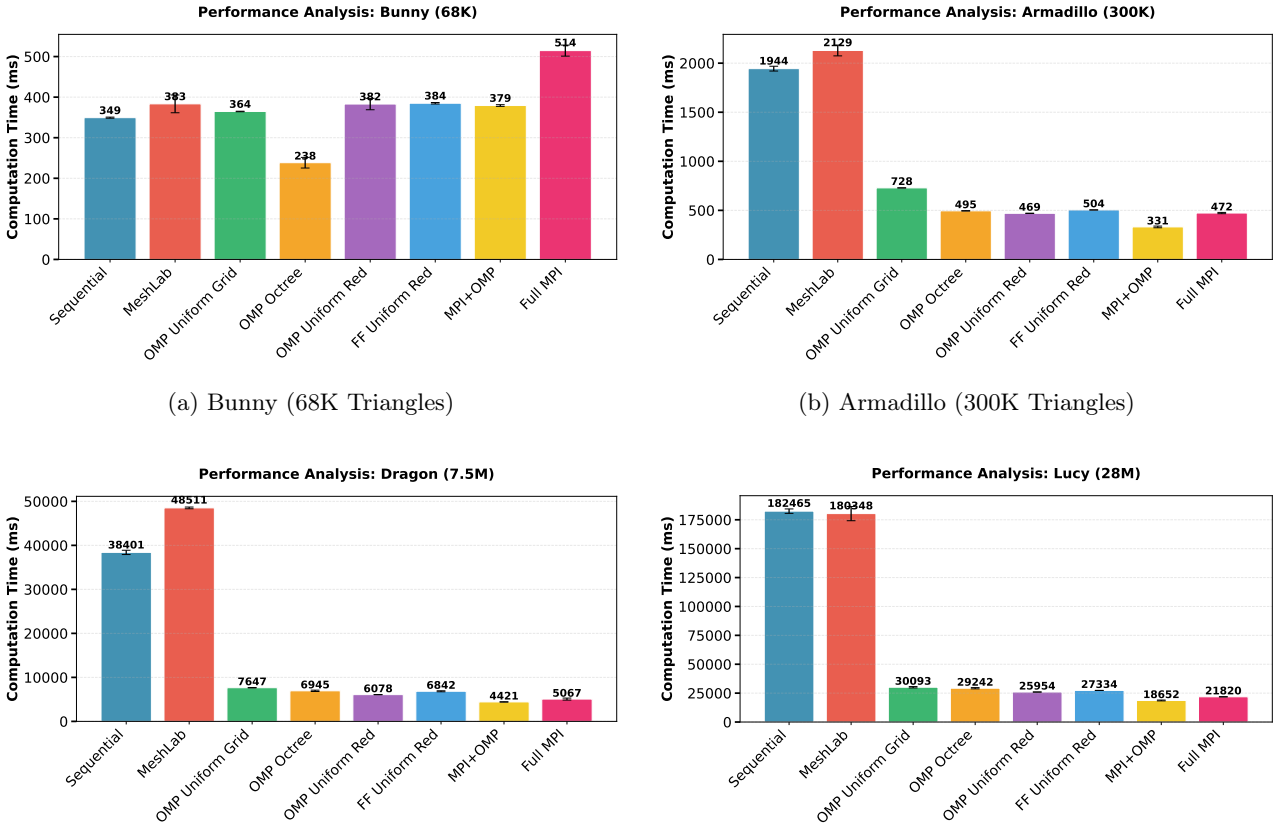
This formulation ensures that, regardless of the number of threads or nodes, the distribution preserves geometric coherence, minimizing data dispersion across distant nodes.

4 Experimental Results Analysis

This section presents a performance analysis of the various parallel implementations developed, examining both the *Shared Memory* and *Distributed Memory* solutions. The analysis was conducted on an HPC cluster consisting of 8 compute nodes, each equipped with 32 logical cores (16 physical cores with Hyper-Threading) and 126 GB of RAM.

Specifically, the *Shared Memory* environment evaluations were performed on a single cluster node, testing four distinct variants: the OpenMP implementation on a *Uniform Grid* (both basic and optimized via the *Hierarchical Reduction* strategy 2.3), the OpenMP variant based on an *Octree* structure, and finally the implementation based on the FastFlow framework. Simultaneously, for the *Distributed Memory* context, the benchmarks leveraged the scalability across the 8 available nodes to evaluate two architectures: a hybrid MPI+OpenMP solution based on the *Master-Worker* paradigm and a *Pure MPI* version founded on the *Distributed Data-Parallel* approach.

The first analysis performed evaluated execution times in absolute terms relative to the sequential state of the art. To this end, the performance of the parallel algorithms, configured to deliver the maximum available computing power, was compared both with a reference sequential implementation and with the decimation algorithm integrated into the *MeshLab* software [CMRC] [4], with the aim of quantifying the overall *speedup* obtained.



(a) Bunny (68K Triangles)

(b) Armadillo (300K Triangles)

(c) Dragon (7.5M Triangles)

(d) Lucy (28M Triangles)

Figura 8: Analysis of execution times (in milliseconds) for the different implementations. The graphs show average performance with relative standard deviation on four models of increasing complexity: (a) Bunny, (b) Armadillo, (c) Dragon, and (d) Lucy.

[4] For practical reasons, this implementation was tested on my PC as it is implemented in software with a GUI.

Observing the graphs in Figure 8, it clearly emerges that the effectiveness of parallel solutions is strictly correlated with the complexity of the input model. For small meshes, such as the *Bunny* model (68K faces), no significant improvements are recorded; on the contrary, in some cases, the parallel and distributed implementations show lower performance compared to their sequential counterparts. This behavior is attributable to the overhead of thread management and communication, which becomes predominant when the computational load is low.

The scenario changes radically when analyzing high-density polygonal models. Examining the most complex mesh, *Stanford Lucy* (28 million faces) [Sta96], a notable speedup is observed compared to the sequential implementation: approximately 7× for the best *Shared Memory* configuration and up to 10× for the distributed implementation.

However, these results, while positive in terms of absolute time, do not provide information regarding the actual scalability of the algorithms. The current comparison pits a single-core (sequential) execution against configurations using 32 cores (Shared Memory) or up to 256 cores (Distributed). In particular, it is noted that the distributed approach, despite using significantly more resources, does not guaranty a proportional performance increase compared to the shared memory version. Therefore, subsequent sections will conduct a more in-depth analysis focused on the efficiency and scalability of the different architectures.

4.1 Shared Memory Analysis

This section analyzes the *strong scaling* of the shared memory implementations. The objective is to evaluate how computation time varies as the number of allocated cores increases while keeping the problem size constant. The *Stanford Lucy* model was selected as the reference input, analyzing the variation in execution times as the number of computing units increases.

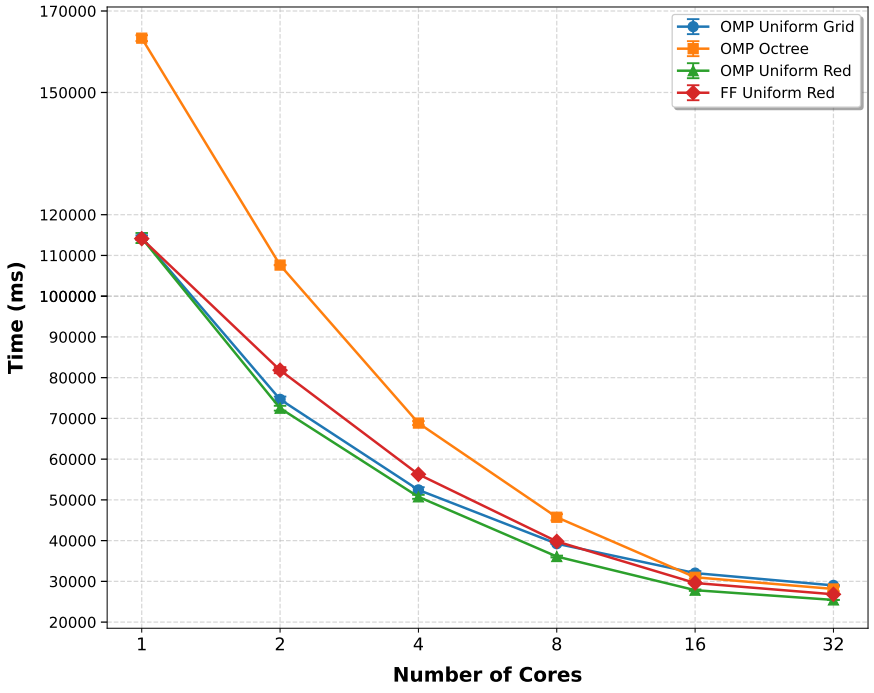


Figura 9: Strong Scaling analysis of Shared Memory implementations on a single node. The graph shows the trend of computation times as the number of physical cores used varies (from 1 to 32).

As seen in Figure 13, the curves show a non-linear decreasing trend: as cores increase, the performance improvement is very pronounced in the initial phases (from 1 to 8 cores) and then progressively levels off toward 32 cores. To accurately analyze the causes of this behavior, we examine the execution times of the individual program components:

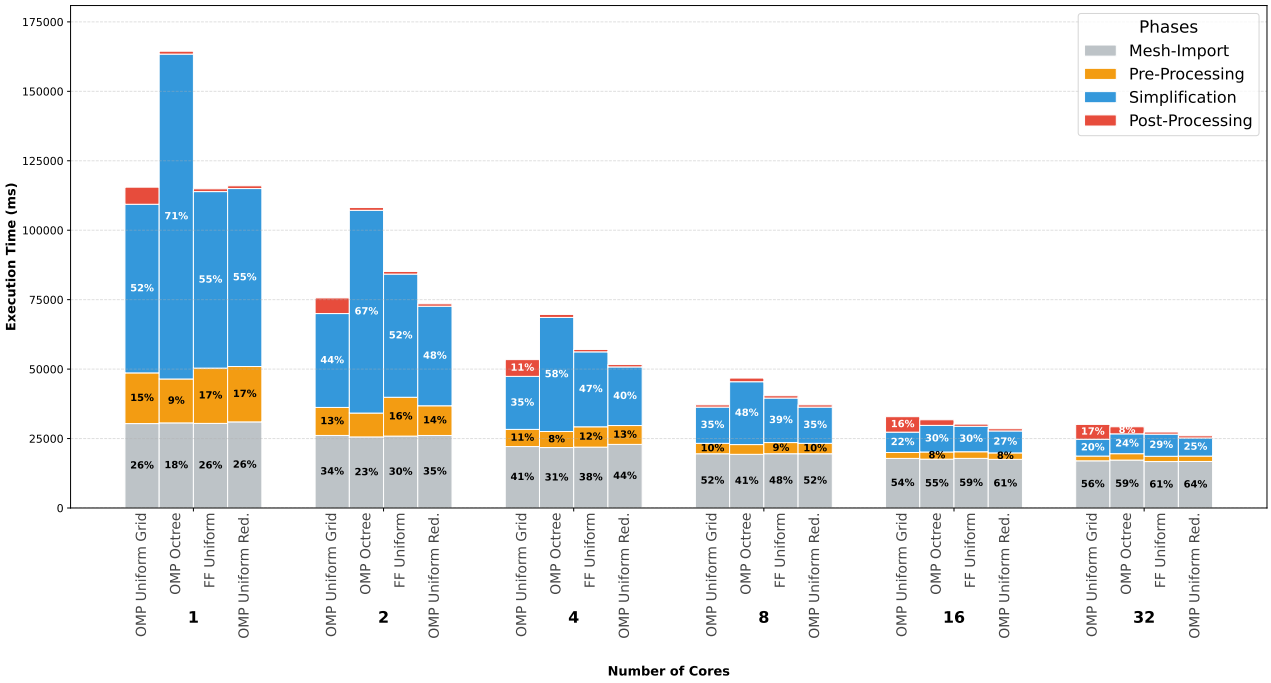


Figura 10: Analysis of execution times for the four implementations analyzed as the number of cores varies. For each configuration, the bars show the total time (top, in ms) and the percentage breakdown into the four main phases: *Mesh-Import*, *Pre-Processing*, *Simplification*, and *Post-Processing*.

The graph in Figure 10 illustrates the breakdown of the operational phases executed in a multi-threaded environment, with the exception of the *Post-processing* phase, which remains a purely sequential operation.

In an ideal scalability scenario, the relative proportions between the different phases should remain constant as the number of cores increases; this would indicate that parallelism is accelerating every section of the code homogeneously. However, the analysis of the graph reveals different trends that deserve attention:

- **Impact of Mesh-import:** It is noted that the percentage of time dedicated to importing the mesh tends to grow progressively as the number of cores increases. Since the absolute loading time remains almost unchanged, this phase becomes a relative bottleneck, suggesting the presence of an *I/O bound* limit or one related to memory bandwidth.
- **Post-processing Weight:** This phase maintains a significant impact across all configurations. The phenomenon is particularly evident in the *Uniform Grid* implementation without *Reduction*, where the need to perform a final sequential refinement pass significantly penalizes the overall speedup, becoming the dominant component in 32-core configurations.

This performance saturation behavior is due to a combination of three determining factors:

1. **Amdahl's Law:** The existence of inherently sequential code portions (such as data loading, structure initialization, or final refinement) limits the theoretical maximum speedup obtainable.

Beyond a certain threshold, adding more cores does not produce significant benefits because the execution time is dominated by the non-parallelizablefraction.

2. **Synchronization Overhead:** As the number of threads increases, the time spent managing synchronization primitives (locks, barriers, and atomic operations) grows proportionally. This inter-thread communication overhead progressively reduces the efficiency of parallelism.

3. **Memory Bound:** Since execution is confined to a single node, the 32 threads compete for access to the same RAM memory bandwidth. The saturation of the memory bus creates a structural bottleneck in accessing mesh data, preventing cores from operating at their maximum theoretical frequency.

4.2 Distributed Memory Analysis

This section analyzes the strong scaling of the distributed implementations. The reference test uses the 28-million-face "Stanford Lucy" mesh as input, evaluating system behavior as the number of computational processes increases.

In both implementations, it is assumed that the minimum configuration includes 32 cores on the *Master* node, dedicated to reading the mesh, performing preliminary operations, and saving the final results. The analysis thus focuses on evaluating how execution times vary as the *Worker* units iincreasease, while the computing capacity of the main node remains fixed.

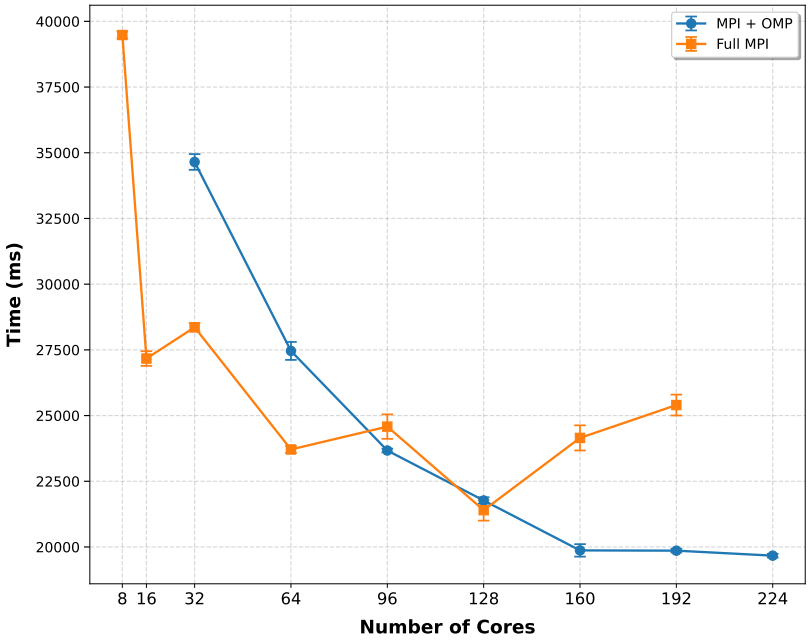


Figura 11: Comparison of strong scaling between the hybrid MPI+OpenMP implementation and the Full MPI version on a dataset of 28M faces.

The analysis of the graph in Figure 13 highlights significantly divergent behaviors between the hybrid Master-Worker implementation and the Full MPI version.

4.2.1 MPI + OpenMP (Hybrid)

In this variant, the increase in computational resources was one *Worker* per iteration; since each *Worker* process internally manages 32 OpenMP threads, the increase in the graph is reported in steps of 32

cores.

Consistent with observations in the shared memory section, there is a clear reduction in total time when moving from 1 to 2 *Workers* (from 32 to 64 cores). However, further increasing the compute units causes performance to saturate around the 20-second threshold, achieving almost no improvement when moving from 160 to 224 cores. This phenomenon indicates that the system has reached a strong scalability limit for the mesh under examination.

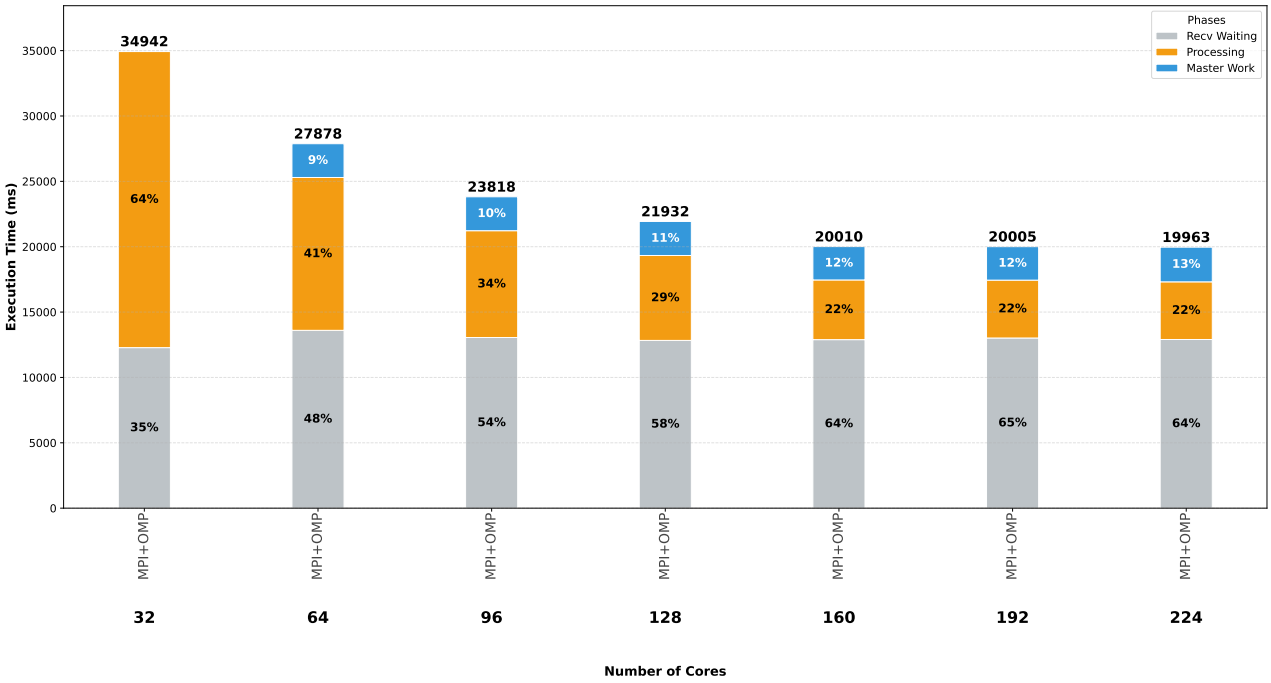


Figura 12: Execution time breakdown for the hybrid MPI+OpenMP implementation as cores increase (32-224). The graph highlights the reduction in effective calculation time compared to the rigidity of waiting and coordination times.

The graph in Figure 12 illustrates the temporal breakdown divided into three main components: the time *Workers* spend waiting to receive data (*Recv Waiting*), the actual computation time (*Processing*), and the finalization time handled by the *Master* (*Master Work*).

Analyzing the data in detail, it is observed that the *Processing* share is drastically reduced (from 22.6s to approximately 4.4s), proving that the internal parallelization logic and load subdivision are efficient. However, the *Recv Waiting* component remains almost constant or slightly increases, stabilizing around 13 seconds. This phase, which includes reading the mesh from disk, partitioning, and the serial distribution of buffers from the *Master* to the *Workers*, constitutes the true *I/O bottleneck* of the application.

Finally, the *Master Work* component, which includes cell merging operations and final data export, shows a slight increase as *Workers* increase, reflecting the greater burden of synchronization and reassembling partitions. In conclusion, the data highlight how, despite the use of asynchronous communications, the inherently sequential nature of I/O and initial distribution prevents the hybrid approach from scaling linearly beyond 128 cores.

4.2.2 Full MPI

The main goal of the *Full MPI* implementation is to mitigate the communicative bottleneck that emerged in the hybrid approach. The strategy adopted is based on two fundamental principles:

1. **Granularity Reduction:** Decreasing partition size to attempt greater parallelization and overlap of communication and calculation phases.
2. **Distributed Communication:** Adopting a scheme where *Workers* take an active role in managing data flow, acting as both a *Collector* for the current stage and an *Emitter* for subsequent ones.

This approach aims to offload the *Master* node from the task of centrally managing every single transaction with the workers and from having to perform the entire final *merge* process on massive memory chunks, distributing part of this complexity across the network.

However, the analysis of the graph in Figure 13 shows that this strategy did not produce the expected improvements. On the contrary, as the number of cores increases, performance not only fails to improve linearly, but in some configurations (particularly those that are not powers of two), it undergoes evident degradation. One of the causes of this instability is found in the *Load Imbalance* resulting from domain decomposition. The tests were carried out by partitioning the space into a $16 \times 16 \times 16$ uniform grid, generating a total of 4096 elementary cells (tasks). For the system to be efficient, the number of cells must be equally divisible by the number of active processes:

- With 16, 32, or 128 processes, 4096 is perfectly divisible, ensuring that each worker receives exactly the same number of cells.
- With configurations such as 96, 192, or 224 processes, the division produces a remainder. This leads to a scenario in which some processes must process more cells than others, forcing the entire cluster to wait for the most heavily loaded and reduce the benefits of additional parallelism.

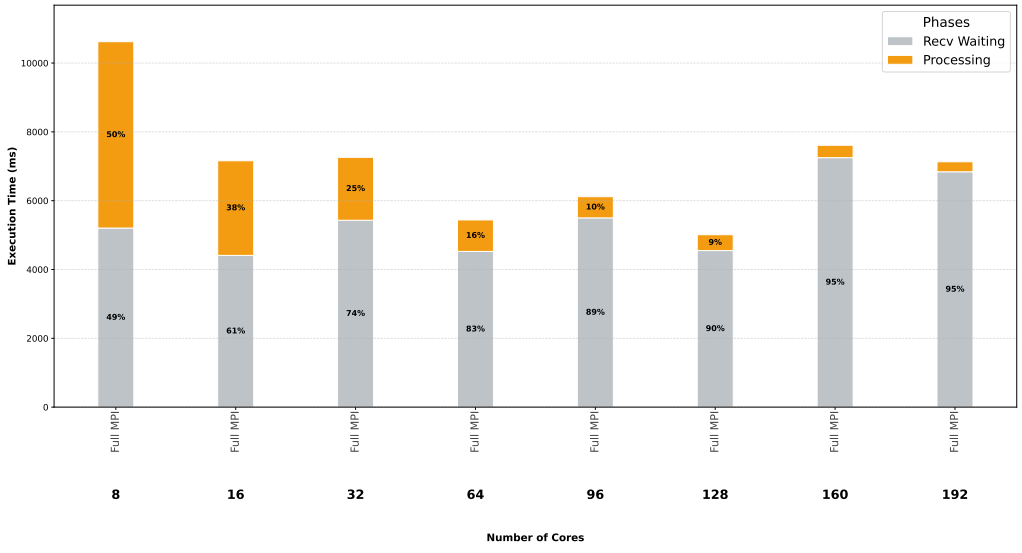


Figura 13: Strong Scaling an alysis for the Full MPI implementation (mesh: stanford_dragon). The graph highlights the clear separation between computation time (*Processing*), which scales linearly up to 196 cores, and communication wait time (*Recv Waiting*), which becomes the dominant bottleneck beyond 128 cores, leading to an inversion of the total time curve.

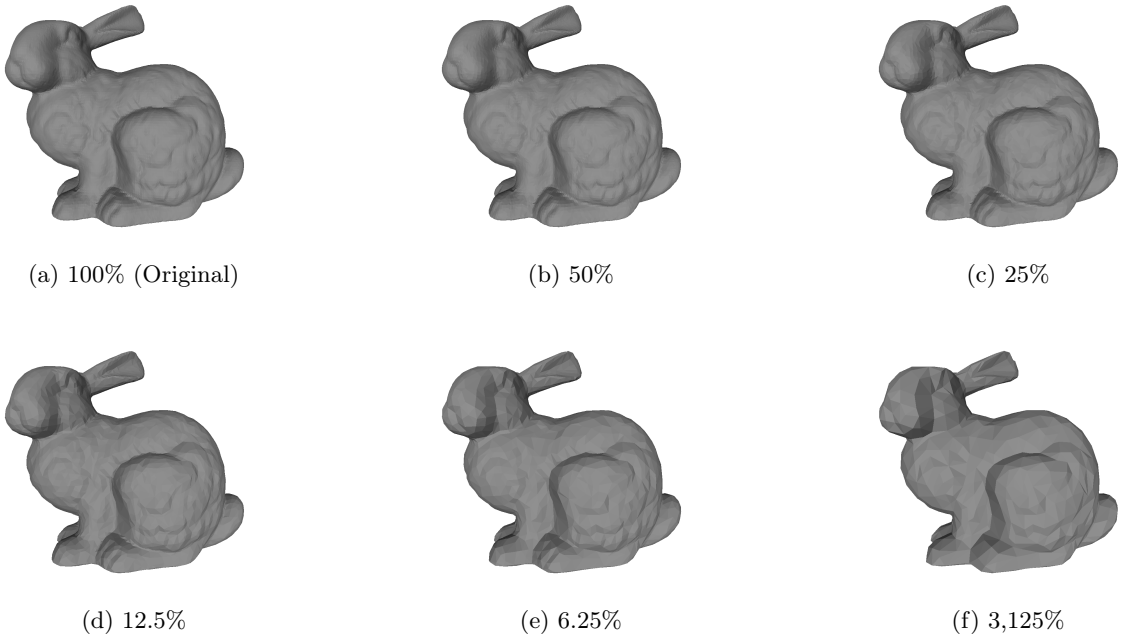
Unlike the previous analyses, this specific evaluation in figure 13 was conducted using the 7-million-face *Stanford Dragon* mesh, due to a lack of data for other models. The proportions in execution times, however, remain similar. If we analyze the communication times, we can see that:

- **Calculation Efficiency (Processing):** Pure computation time shows almost ideal scalability. Moving from 32 cores ($\approx$ 1829 ms) to 128 cores ($\approx$ 460 ms), the time is reduced by about 4 times, demonstrating that decomposition into 4096 elementary cells is effective in distributing the computational load.
- **Dominance of Recv Waiting:** Despite the calculation scaling well, the total time is dominated by the *Recv Waiting* component. Even at 32 cores, the wait for message reception ($\approx$ 5831 ms) is three times the computation time. This phenomenon explodes in the most extreme configurations: at 160 and 196 cores, communication overhead accounts for over 95% of total execution time.
- **Sweet Spot:** The best overall time was recorded at **128 cores**. Beyond this threshold, the system enters a *negative scaling* regime: further domain fragmentation increases the frequency and number of MPI messages exchanged, saturating the interconnection between nodes.
- **Effect of non-power-of-two configurations:** In the 160 and 196 core configurations, waiting times are not only high but also extremely unstable (7.25 s and 6.84 s respectively). This confirms how the *Load Imbalance* generated by the remainder of the division between the 4096 tasks and the active processes creates synchronization bottlenecks, where most workers remain idle waiting for the final processes engaged in processing residual tasks.

In conclusion, the *Full MPI* strategy for this specific mesh fails to scale beyond 128 processes. The cost of distributed chunk management and the intrinsic latency of the MPI network for small messages make the allocation of additional computational resources counterproductive.

5 Final Output

The following series of images [14] illustrates the algorithm's performance by progressively simplifying the *Stanford Bunny*, halving the face count at each step. This process is executed in parallel while maintaining excellent visual quality.



(a) 100% (Original) (b) 50% (c) 25%

(d) 12.5% (e) 6.25% (f) 3,125%

Figura 14: Progression of geometric simplification on the *Stanford Bunny* model using the QEM algorithm. The sequence illustrates the iterative reduction of the face count by halving the polygonal budget at each stage, demonstrating the algorithm's ability to preserve visual fidelity and primary topology even at drastic simplification thresholds.

Riferimenti bibliografici

[ASCE02] Nicolas Aspert, Diego Santa-Cruz, and Touradj Ebrahimi. MESH: Measuring errors between surfaces using the hausdorff distance. In *IEEE International Conference on Multimedia and Expo (ICME)*, volume 1, pages 705–708, Lausanne, Switzerland, August 2002.

[CMRC] Paolo Cignoni, Alessandro Muntoni, Guido Ranzuglia, and Marco Callieri. Meshlab.

[CMRS03] Paolo Cignoni, Claudio Montani, Claudio Rocchini, and Roberto Scopigno. External memory management and simplification of huge meshes. *IEEE Transactions on Visualization and Computer Graphics*, 9(4):525–537, October 2003.

[CMS98] P. Cignoni, C. Montani, and R. Scopigno. A comparison of mesh simplification algorithms. *Computers Graphics*, 22(1):37–54, 1998.

[GH97] Michael Garland and Paul S. Heckbert. Surface simplification using quadric error metrics. In *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '97, page 209–216, USA, 1997. ACM Press/Addison-Wesley Publishing Co.

[Sta96] Stanford Computer Graphics Laboratory. The stanford 3D scanning repository. http://graphics.stanford.edu/data/3Dscanrep/, 1996. Accessed: January 30, 2026.